Mini Search Engine: Project Documentation

Course: Web Search and Mining (WSM)

Submitted by:

- **Shreyansh Mishra , B124124**
- **Sohan Bal , B124131**
- **Swetarth Yadav , B124143**

Stack: Python (Flask) backend, React (Vite) frontend

1. Overview

This project is a small web search engine built to apply the techniques taught in the WSM course on a collection of 15 text documents about information retrieval. It offers two search interfaces: a **basic boolean search** and an **advanced ranked search** with query expansion and relevance feedback. Every result carries a relevance score, and the system includes index compression with measurable savings.

The backend is deliberately kept simple: plain Python dictionaries and lists, no external search libraries, so each algorithm can be read and explained directly.

2. Objectives

- Build an inverted index and answer boolean, phrase and ranked queries.
- Improve search results and address low recall using both local methods (relevance feedback) and global methods (thesaurus, query log).
- Reduce index size using dictionary and postings compression.
- Present everything through a simple React interface.

3. Architecture

Layer	Technology	Role
Frontend	React + Vite	Two search boxes, results with scores, feedback buttons, index statistics
Backend API	Python + Flask	Seven REST endpoints, all JSON
Search engine	Pure Python (`engine.py` , `compression.py`)	Indexing, query processing, ranking, feedback
Data	15 `.txt` files, `thesaurus.json`	Document collection and manual thesaurus

The frontend sends requests to `/api/...`, which the Vite dev server forwards to the Flask server on port 5000. The index is built in memory when the backend starts.

4. Project structure

```
search-engine/
  backend/
    app.py            Flask API routes
    engine.py         index, boolean parser, soundex, ranking, feedback,
query log
    compression.py    dictionary compression, variable-byte postings
    thesaurus.json    manual thesaurus (14 synonym groups)
    requirements.txt
    documents/        15 text documents (first line = title)
  frontend/
    index.html, package.json, vite.config.js
    src/ main.jsx, App.jsx, ResultCard.jsx, App.css
  README.md
```

5. Document collection

15 documents covering information retrieval topics: introduction, boolean model, inverted index, positional index and phrases, dictionary and postings compression, tolerant retrieval and Soundex, vector space model, relevance feedback and Rocchio, pseudo and indirect feedback, query expansion and thesauri, query log mining, evaluation, web crawling and link analysis, and text normalization. More documents can be added by placing `.txt` files in `backend/documents/` (first line is the title) and restarting the backend.

Index size for this collection: **15 documents, 468 distinct terms, 854 postings, 1053 distinct biwords**.

6. Features

6.1 Text processing and indexing

- **Lowercasing and tokenizing.** All text and queries are lowercased and split into alphanumeric tokens, so "Search" and "search" are identical. Documents and queries go through the same processing.
- **Positional inverted index.** For every term, the index stores each document containing it and the positions of the term in that document.
- **Biword index.** Every pair of consecutive words is stored as one entry, mapped to the documents containing it.
- **Soundex index.** Every dictionary term is grouped under its four-character Soundex code.

6.2 Basic search: boolean retrieval

- Operators **AND, OR, NOT** and **brackets**, in upper or lower case.
- Precedence: brackets, then NOT, then AND, then OR. Two terms with no operator are treated as AND.
- **Phrase queries** in quotes, for example `"relevance feedback"`.
- Matching documents are found by combining the postings of each term as sets. Matches are then **ranked with tf-idf cosine scores** on the positive (non-negated) terms.
- The interface explains what happened, such as the lowercasing applied or which index answered a phrase.

- Invalid queries (empty query, missing bracket, dangling operator) return a clear error message.

6.3 Phrase and biword searching

Phrase length	Method used
1 word	Inverted index
2 words	Biword index (direct lookup)
3 or more words	Positional index: intersect documents, then check that the words occur at consecutive positions

6.4 Soundex

Soundex converts a word to its first letter plus three digits, so words that sound alike share a code (Robert and Rupert both give R163; Smith and Smyth both give S530). In advanced search, up to 5 terms with the same code are added to the query. Words not found in the index (likely misspellings) get full weight; words that exist get a lower weight of 0.4.

6.5 Ranking and scores

The ranking uses the vector space model.

- **Term weight = (1 + log10 tf) × idf, where idf = log10(N / df).**
- Document and query vectors are length-normalised.
- Score = cosine similarity, between 0 and 1, shown next to every result.

6.6 Dictionary and postings compression

- **Dictionary as a string with blocking (k = 4).** All terms are stored in one string, each preceded by a length character. Only one pointer is kept per block of 4 terms. A lookup does a binary search over the blocks, then a short scan inside one block. The position of a term in the sorted list is its term id.
- **Postings compression.** Document ids are stored as gaps between consecutive ids and encoded with variable-byte encoding.
- Both structures are used by the search itself: every term lookup goes through the compressed dictionary and decodes the compressed postings.

Structure	Before	After	Saved
Dictionary	6552 bytes (fixed-width terms)	3752 bytes	42.7%
Postings	3416 bytes (4 bytes per doc id)	854 bytes	75.0%

These figures are also shown live in the "Index and compression" tab.

6.7 Query expansion (global methods)

Available as checkboxes in the advanced search:

- **Manual thesaurus.** `thesaurus.json` holds 14 groups of synonyms (for example search, retrieval, query, lookup). Synonyms of the query words are added with weight 0.5.
- **Query log mining.** The system keeps a log of advanced queries, starting from 10 seed queries and growing with use. It finds terms that appear in the same past queries as the user's words and adds the top 3 (weight 0.3). It also shows **query suggestions** from past queries that share a word with the current one.

Added terms are displayed as chips under the search box, with the reason as a tooltip.

6.8 Relevance feedback (local methods)

All feedback methods re-rank the results with an improved query vector.

Method	How it works
Explicit feedback (Rocchio)	The user marks results as Relevant or Not relevant and presses "Search again with my feedback". New query = 1.0 × old query + 0.75 × centroid of relevant documents − 0.15 × centroid of non-relevant documents. Negative weights are dropped, and up to 6 new terms are added.
Probabilistic feedback	Binary independence model. For each term in the relevant documents, p = (vr + 0.5) / (R + 1), u = (df − vr + 0.5) / (N − R + 1), and weight c = log10(p(1 − u) / (u(1 − p))). The top 6 positive terms are added.

Method	How it works
Pseudo relevance feedback	The top 3 results are assumed relevant and Rocchio is applied automatically, with no user effort.
Indirect feedback	Opening a result is recorded as a click for that query. With "Use past clicks" ticked, the 3 most-clicked documents for the same query are treated as relevant.

When more than one source is available, the priority is: explicit marks, then past clicks, then pseudo feedback.

7. User interface

The React interface has three tabs:

1. **Basic search.** One box for boolean queries, with hints for the syntax.
2. **Advanced search.** One box for ranked queries, plus checkboxes for Soundex, thesaurus, query log, pseudo feedback and click feedback, and a selector for Rocchio or probabilistic feedback.
3. **Index and compression.** Collection statistics and the compression table.

Each result shows the title, score, a snippet containing the query terms and the file name. Clicking a title opens the full document. In advanced mode, each result has Relevant and Not relevant buttons, shown with a green or red edge.

8. API reference

Method	Endpoint	Purpose
GET	`/api/documents`	List all documents
GET	`/api/documents/<id>`	Full text of one document
POST	`/api/boolean`	Boolean search. Body: `query`
POST	`/api/advanced`	Ranked search. Body: `query`, `options`, `relevant`, `nonrelevant`

Method	Endpoint	Purpose
POST	`/api/click`	Record a click. Body: `query`, `doc_id`
GET	`/api/suggest?q=`	Query suggestions from the log
GET	`/api/stats`	Index and compression statistics

Responses contain the mode, total hits, the result list (id, title, score, snippet, filename), the explanation steps, the expanded terms and the suggestions. Errors return HTTP 400 with a message.

9. How to run

1. Install Python 3.9 or newer and Node.js.
2. Backend: `cd backend`, `pip install -r requirements.txt`, `python app.py` (runs on port 5000).
3. Frontend, in a second terminal: `cd frontend`, `npm install`, `npm run dev`, then open the printed address (http://localhost:5173).

The files `query_log.json` and `clicks.json` are created in `backend/` when the system is used.

10. Example queries

Tab	Query	What it shows
Basic	`query AND (expansion OR feedback) NOT thesaurus`	Boolean operators, brackets, lowercasing, scores
Basic	`"relevance feedback"`	Biword phrase search
Basic	`"inverted index construction"`	Positional phrase search
Advanced	`Rupert` with Soundex	Finds the document mentioning Robert
Advanced	`relevence feedbak` with Soundex	Handles misspellings
Advanced	`search recall` with Thesaurus, Query log, Pseudo feedback	Expansion and automatic re-ranking

Tab	Query	What it shows
Advanced	Any query, then mark results and search again	Rocchio or probabilistic feedback
Advanced	Open the results for `rocchio` a few times, then search `rocchio` with "Use past clicks"	Indirect feedback

11. Mapping to the syllabus

Syllabus topic	Status	Implementation
Addressing low recall	Covered	Expansion and feedback features
Global vs. local methods	Covered	Global: thesaurus, query log. Local: relevance feedback
Vector space model and Rocchio	Covered	`rank()`, `rocchio()`
Probabilistic relevance feedback	Covered	`probabilistic()`
Pseudo relevance feedback	Covered	Top 3 results assumed relevant
Indirect relevance feedback	Covered	Click recording and click-based feedback
Query expansion and reformulation	Covered	Thesaurus, soundex, query log and feedback terms
Manual thesaurus / controlled vocabulary	Covered	`thesaurus.json`
Automatic thesaurus generation	Partial	Query log co-occurrence only, not collection co-occurrence
Query log mining	Covered	Co-occurrence expansion and suggestions
Boolean search, lowercasing	Covered	`BooleanParser`, `tokenize()`
Soundex	Covered	`soundex()`

Syllabus topic	Status	Implementation
Phrase and biword search	Covered	Biword and positional indexes
Dictionary compression	Covered	`compression.py`
Evaluation strategies	Not implemented	See limitations

12. Limitations and future work

- The collection is small (15 documents), so results show the techniques rather than web-scale performance. A real web search engine would also need crawling, link analysis, and distributed indexing.
- No stemming or stop-word removal; "connect" and "connected" are different terms.
- The index is held in memory and rebuilt at startup.
- Evaluation (precision, recall, mean average precision, residual collection) is not implemented and would need judged test queries.
- Possible extensions: an automatic thesaurus built from document co-occurrence, spelling correction with edit distance, wildcard queries, and a document upload page.